GAME PROGRAMMING COURSE — C# BASICS

命名のルールと注意点

NAMING

変数・関数・クラスに「読める名前」をつけるための細かいルール

対象：1年生

PART 1

名前はなぜ大事か

同じ処理でも、名前だけで読みやすさが大きく変わる

1

名前の役割

同じ処理、違うのは名前だけ

C# PlayerAttack.cs — BEFORE

```
if (f && c > t) D();
```

C# PlayerAttack.cs — AFTER

```
if (isGrounded && comboCount > comboLimit) DoFinisher();
```

- BEFORE は、f・c・t・D が何かを調べに行かないと読めない
- AFTER は「地面にいて、コンボ数が上限を超えたらフィニッシュ技」とそのまま読める
- 名前は、コメントを書かなくても伝わる**いちばん身近な説明書**

POINT 名前を読むのは、先生・チームの仲間・1か月後の自分。自分以外の人に向けて名前をつける。

悪い名前の典型

よく見る「伝わらない名前」

NG の名前	何が問題か	直した例
a ・ x ・ n	何が入っているか、まったく分からない	hp ・ moveX
tmp ・ temp	「一時的な何」なのかが分からない	previousPosition
data ・ info	どんな変数にも当てはまってしまう	highScores
flg ・ flag ・ check	true のときに何を意味するのか分からない	isGameOver
Func1 ・ Method	何をする関数なのか分からない	SpawnEnemy
enemy2 ・ obj3	番号が「何の違い」なのか分からない	bossEnemy
hoge ・ test ・ aaa	試しにつけた仮の名前がそのまま残っている	意味のある名前
NewBehaviourScript	Unity が自動でつけた名前のまま使っている	PlayerMovement

POINT 共通点は「書いた本人にしか分からない」こと。書いた直後は分かっても、数日で本人も分からなくなる。

PART 2

書き方のルール

大文字・小文字の使い分けと、C# が決めているルール

2

ケースの種類

単語のつなぎ方は4種類

C# で使う

PascalCase（パスカルケース）

`PlayerState` ・ `TakeDamage`

すべての単語の先頭を大文字にする。クラス・メソッドなどに使う。

C# で使う

camelCase（キャメルケース）

`moveSpeed` ・ `enemyCount`

最初の単語だけ小文字で、あとの単語の先頭は大文字。変数・引数に使う。

C# では基本使わない

snake_case（スネークケース）

`move_speed` ・ `enemy_count`

単語を _ でつなぐ。Python などの言語でよく使われる書き方。

C# で使う（定数）

UPPER_SNAKE_CASE

`MAX_HP` ・ `CLEAR_SCORE`

すべて大文字を _ でつなぐ。この学校では定数（const）に使う。

C# の使い分け

何に、どのケースを使うか

種類	書き方	例
クラス・構造体	PascalCase	Player ・ EnemySpawner
列挙体の型名と値	PascalCase	PlayerState.Idle
メソッド（関数）	PascalCase	TakeDamage() ・ Jump()
プロパティ	PascalCase	MaxHp ・ IsAlive
定数（const）	UPPER_SNAKE_CASE	CLEAR_SCORE ・ MAX_HP
インターフェース	I ＋ PascalCase	IDamageable
private フィールド	camelCase	moveSpeed ・ jumpPower
ローカル変数・引数	camelCase	damage ・ enemyCount

POINT 「大文字で始まる ＝ 型やメソッド」「小文字で始まる ＝ 変数」「全部大文字 ＝ 定数」。見ただけで種類の見当がつく。

流儀の違い

フィールドの書き方は流儀が分かれる

Microsoft の規約

`_moveSpeed`

private フィールドの先頭に _ をつける。static なら s_ をつける。

Unity のスタイルガイド

`m_MoveSpeed`

private は m_、定数は k_、static は s_ をつける例がのっている。

この授業の資料

`moveSpeed`

記号はつけず、camelCase だけで書く。Inspector にもそのまま出せる。

- Unity の API は **transform** や **gameObject** のように小文字始まり。C# の規約と違っても戸惑わない
- Unity は Inspector に表示するとき、m_ や _ を外して「Move Speed」のように見せてくれる

POINT どれが正解というより、**1つのプロジェクトの中で統一する**ことがいちばん大事。チームで作るなら最初に決める。

C# の決まり

守らないとエラーになるルール

ルール	NG（エラー）	OK
文字か _ で始める	2ndPlayer	player2 ・ secondPlayer
使える記号は _ だけ	move-speed ・ hp!	moveSpeed
予約語はそのまま使えない	class ・ int ・ new ・ event	playerClass ・ eventName
スペースは入れられない	move speed	moveSpeed

エラーにはならないが、気をつけること

- **大文字と小文字は別物**。hp・Hp・HP は3つの別の変数になる。1つの書き方にそろえる
- **日本語の名前**（int 体力;）も書けるが使わない。入力の切り替えや全角・半角の見間違いが起きやすい
- **_ を2つ続ける名前**（__count）はコンパイラが使う名前なので避ける
- **@class** のように @ をつけると予約語も使えるが、普段のコードでは使わない

PART 3

種類ごとの付け方

3

変数・bool・メソッド・クラス・列挙体、それぞれのコツ

変数

変数は「何が入っているか」を名詞で

観点	NG	OK	理由
何の値か	num ・ value	enemyCount	「何の」数なのかを入れる
数は Count	enemyNum	enemyCount	Num は「番号」とも読める
1つか複数か	enemy（配列）	enemies	複数形なら入れ物だと分かる
単位を入れる	delay	delaySeconds	秒なのかフレームなのか
最大と現在	hp ・ hp2	maxHp ・ currentHp	対になる名前でそろえる
何の位置か	pos2	spawnPosition	どこの位置なのかを入れる
1つ取り出したもの	e	enemy	foreach の変数は単数形

```
foreach (Enemy enemy in enemies) { enemy.TakeDamage(10); }
```

bool

bool は「はい／いいえ」で答えられる名前に

is

〜の状態である

`isDead`
`isGrounded`

has

〜を持っている

`hasKey`
`hasShield`

can

〜できる

`canJump`
`canAttack`

should

〜すべき

`shouldRespawn`
`shouldShowTutorial`

否定形の名前にしない

isNotDead ではなく isAlive。
if (!isNotVisible) のような二重否定は、読む人が頭の中で2回ひっくり返すことになる。

true の意味を1つに決める

bool jump は「ジャンプ中？」「押した？」「できる？」のどれか分からない。
→ isJumping ・ jumpPressed ・ canJump

メソッド

メソッドは「何をするか」を動詞から

動詞	意味	例
Get	値を取り出す	GetScore()
Set	値を設定する	SetVolume(0.5f)
Add ・ Remove	加える・取り除く	AddItem(item) ・ RemoveItem(item)
Spawn	ゲーム内に生み出す	SpawnEnemy()
Update	最新の状態にする	UpdateHpBar()
Calculate	計算して求める	CalculateDamage(attack)
Reset	最初の状態に戻す	ResetPosition()
Is ・ Can ・ Has	bool を返す（質問の形）	IsGameOver() ・ CanJump()

NG 例 Player()・Score() は動詞がなく、何をするか分からない。DoSomething()・Process() は何にでも当てはまってしまう。

対になる言葉

反対の処理は、決まった組み合わせで

Show / Hide 表示する・隠す	**Open / Close** 開く・閉じる	**Add / Remove** 加える・取り除く	**Start / Stop** 始める・止める
Begin / End 開始・終了	**Enable / Disable** 有効にする・無効にする	**Save / Load** 保存する・読み込む	**Lock / Unlock** ロックする・解除する
Increase / Decrease 増やす・減らす	**Min / Max** 最小・最大	**First / Last** 最初・最後	**Get / Set** 取り出す・設定する

NG 例 ShowMenu() と DeleteMenu()、OpenDoor() と FinishDoor() のように組み合わせを崩すと、対の処理だと気づけない。

クラス・スクリプト

クラスは「何をするか」まで分かる名前に

OK

```
PlayerMovement
PlayerHealth
EnemySpawner
ScoreCounter
```

「誰の」＋「何をするか」で、名前を見れば担当している処理が分かる。

NG

名前	問題
Player ・ PlayerScript	プレイヤーの「何」を担当するか分からない
Player2 ・ PlayerNew	何が違うのか分からない
Manager	何を管理するのか分からない
GameManager に全部	役割が広すぎて何でも入る箱になる

Unity の決まり：ファイル名とクラス名は同じにする

PlayerMovement.cs の中身は **public class PlayerMovement : MonoBehaviour**。ずれていると「the file name and class name match」と言われ、GameObject にアタッチできない。1つのファイルに MonoBehaviour は1つだけ。

列挙体

列挙体は型名を単数形、値を短く

C# PlayerState.cs BEFORE

```cs
enum PlayerStates
{
    PlayerIdle,
    playerRun,
    PLAYER_JUMP
}
```

C# PlayerState.cs AFTER

```cs
enum PlayerState
{
    Idle,
    Run,
    Jump
}
```

- **型名は単数形**：変数に入る値は1つだけなので PlayerState（PlayerStates にしない）
- **値は PascalCase**：playerRun や PLAYER_JUMP のように書き方を混ぜない
- **値に型名をくり返さない**：PlayerState.PlayerIdle は「Player」が2回。PlayerState.Idle で十分

PART 4

よくある落とし穴

4

略語・ローマ字・名前と中身のずれ・バラバラな呼び方

落とし穴 1 ｜ 略語

略すのは「みんなが知っている略語」だけ

使ってよい略語

`Max` ・ `Min` ・ `Id` ・ `UI` ・ `HP`

`x` ・ `y` ・ `z`（座標）

`i` ・ `j`（for のカウンタ）

ゲームやプログラムで広く使われていて、誰が読んでも同じ意味になるもの。

使わない略語

`plyr` → `player`

`enmCnt` → `enemyCount`

`atkPw` → `attackPower`

自分で作った略語。scr は score？ screen？ script？ と読む人が迷う。

頭文字の略語を名前に入れるときの書き方

- 2文字なら両方大文字：**UIManager** ・ **IOHelper**
- 3文字以上なら先頭だけ大文字：**JsonLoader** ・ **HtmlParser**（JSONLoader にしない）
- maxHp と maxHP はどちらも見かける。どちらにするかをプロジェクトで決めてそろえる

落とし穴 2 ｜ ローマ字とスペル

ローマ字を使わない、スペルを間違えない

ローマ字 → 英語

NG	OK
teki	enemy
tokuten	score
jikan	time ・ timer
kougeki	attack
tekiCount（混在）	enemyCount

よくあるスペルミス

NG	OK
Hight	Height
Lenght	Length
Recieve	Receive
Postion	Position
Enemey ・ Plyaer	Enemy ・ Player

POINT 英単語が分からなければ辞書や翻訳で調べてよい。スペルミスは入力補完でコピーされて広がるので、見つけたらすぐ直す。

落とし穴 3 ｜ 名前と中身のずれ

名前と中身がずれている

名前	実際にやっていること	直し方
GetScore()	返す前にスコアを 10 足している	AddScore() と GetScore() に分ける
isDead	HP が残っているときに true	isAlive にするか、中身を直す
SaveData()	保存したあと、タイトル画面に戻している	保存と画面移動を別のメソッドに
playerSpeed	実は敵の速さが入っている	enemySpeed に変える

なぜ危ないか

意味の分からない名前は「調べよう」と思える。ウソの名前は信じて使ってしまうので、バグの原因に気づけない。

どうしてずれるか

最初は合っていたのに、あとから処理を足したり使い方を変えたりしたとき。**中身を変えたら、名前も見直す**。

落とし穴 4 ｜ 呼び方のバラつき

同じものは、同じ言葉で呼ぶ

意味	バラバラな例（1つのプロジェクト内）	そろえた例
敵	enemy ・ teki ・ enemyObj ・ enemyGO	enemy
今の HP	currentHp ・ nowHp ・ curHp ・ hpNow	currentHp
消す	DestroyEnemy ・ KillBoss ・ DeleteBullet	Destroy にそろえる
点数	score ・ point ・ pt	score

違う言葉は「違う意味」に見える

Kill と Destroy が混ざっていると、読む人は「何か違いがあるのかな」と考えてしまう。

プロジェクトの単語帳を作る

「敵 ＝ enemy」「今の ＝ current」「消す ＝ Destroy」のように、よく使う言葉を最初に決めて書き出しておく。

落とし穴 5 ｜ 名前の長さ

長さは「使われる範囲」で決める

数行だけ

`i`

for 文のカウンタ。すぐ終わるので短くてよい。

→

メソッドの中

`damage`

ローカル変数・引数。メソッド名と合わせて意味が分かればよい。

→

クラス全体

`attackCooldown`

フィールド。どこで読まれても分かるように説明的にする。

長すぎても読みにくい

thePlayerCharacterCurrentHitPointValue は情報が多すぎる。
→ currentHp で十分伝わる。

クラス名をくり返さない

EnemySpawner の中の enemySpawnerInterval は名前が重なる。
→ spawnInterval で十分伝わる。

落とし穴 6 ｜ 型の名前

型の名前を、変数名に入れない

NG	OK	理由
int hpInt	int hp	型は宣言を見れば分かる
string nameString	string playerName	型より「誰の名前か」が大事
Enemy[] enemyArray	Enemy[] enemies	複数形で十分伝わる

型を変えると名前がウソになる

hpInt を float に変えたら、名前だけ int のまま残る。名前は「中身の意味」だけを表す。

Unity のコンポーネントは例外

1つだけなら spriteRenderer のように型名でよい。2つ以上なら bodyRenderer ・ shadowRenderer と役割で分ける。

UI UI の部品は「何の ＋ 部品の種類」にすると分かりやすい：scoreText ・ hpBarImage ・ startButton

PART 5

Unity での名前

5

オブジェクト・文字列・Inspector の注意と、名前の正しい変え方

コード以外にも名前をつける

場所	NG（つけたまま放置）	OK
GameObject	GameObject (1) ・ Cube ・ Square	Player ・ Ground ・ Goal
Prefab	New Prefab ・ Enemy 1	Slime ・ BossDragon
Scene	SampleScene ・ Scene2	Title ・ Stage1 ・ Result
スクリプト	NewBehaviourScript	PlayerMovement
画像・音	image01.png ・ 無題.png	player_idle.png ・ se_jump.wav
フォルダ	全部 Assets の直下	Scripts ・ Prefabs ・ Scenes ・ Sprites

POINT ファイル名に日本語やスペースを入れない。GameObject.Find や Resources.Load は名前で探すので、名前を変えると動かなくなる。

文字列で書く名前は、打ち間違えても気づけない

名前を文字列で指定する場所

```
CompareTag("Enemy")
SceneManager.LoadScene("Stage1")
Input.GetAxis("Horizontal")
animator.SetBool("isJumping", true)
```

- "enemy" と "Enemy" は別の名前
- コンパイルエラーにならず、**実行して初めて**気づく

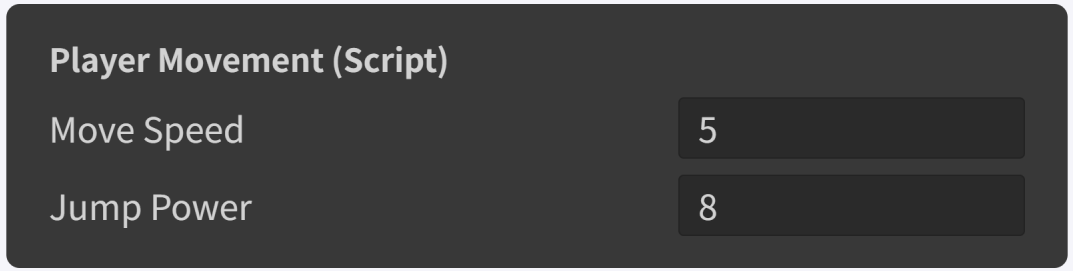
C# Tags.cs　　対策

```
1  public static class Tags
2  {
3      public const string PLAYER = "Player";
4      public const string ENEMY = "Enemy";
5  }
6
7  // 使う側
8  if (other.CompareTag(Tags.ENEMY))
9      TakeDamage(10);
```

POINT　文字列を1か所の定数にまとめると、入力補完が効き、打ち間違えたらコンパイルエラーで教えてくれる。

変数名は Inspector にも表示される

```
[SerializeField] float moveSpeed = 5f;
[SerializeField] float jumpPower = 8f;
```



camelCase で書くと、Unity が単語ごとに区切って表示してくれる。m_ や _ は表示から外れる。

名前を変えると、設定した値が消える

Inspector で 8 に変えておいた speed を moveSpeed に改名すると、Unity は別の変数だと思い、値が初期値の 5 に戻る。

```
using UnityEngine.Serialization;

[FormerlySerializedAs("speed")]
[SerializeField] float moveSpeed = 5f;
```

前の名前を書いておくと、値を引きついでくれる。使わない場合は、改名後に Inspector の値を確認して入れ直す。